

Build Multistage Python Docker Images with uv

Fabian Preiß

About Me

Fabian Preiß

- Technical Co-Founder @ Digon.IO GmbH
- Python developer (12+ years)
- I like: reproducible builds + simple ops

What makes a good production image?

What makes a good production image?

- Fast build time

What makes a good production image?

- Fast build time
- Small image -> fast pulls & startup

What makes a good production image?

- Fast build time
- Small image -> fast pulls & startup
- Reproducibility

What makes a good production image?

- Fast build time
- Small image -> fast pulls & startup
- Reproducibility
- Minimal overhead and dependencies

What makes a good production image?

- Fast build time
- Small image -> fast pulls & startup
- Reproducibility
- Minimal overhead and dependencies
- Few licensing questions

What makes a good production image?

- Fast build time
- Small image -> fast pulls & startup
- Reproducibility
- Minimal overhead and dependencies
- Few licensing questions

We want: **only what we actually run** in the final image

A typical python image

```
1 FROM python:3.13.5-slim-bookworm
2
3 WORKDIR /app
4
5 COPY pyproject.toml uv.lock ./
6 # --locked: fail on outdated lockfile, --no-dev: disable dev deps
7 # --no-install-project: excludes project, improves layer caching
8 RUN pip install --no-cache-dir uv && \
9     uv sync --locked --no-dev --no-install-project
10
11 COPY src/ ./src
12 RUN uv pip install .
13
14 ENV PATH="/app/.venv/bin:${PATH}"
15
16 CMD ["python", "-m", "restapipy"]
```

Build with `docker build -f Dockerfile-simple -t restapipy:simple .`

6 months later: security drift

Images age over time

6 months later: security drift

Images age over time

Naive Debian slim

6 months later: security drift

Images age over time

Naive Debian slim

```
1 trivy image --quiet restapipy:simple | grep "^Total:" -B2
2 restapipy:simple (debian 12.11)
3 =====
4 Total: 106 (UNKNOWN: 1, LOW: 65, MEDIUM: 29, HIGH: 8, CRITICAL: 3)
5 --
6 Python (python-pkg)
7 =====
8 Total: 2 (UNKNOWN: 0, LOW: 0, MEDIUM: 1, HIGH: 1, CRITICAL: 0)
```


6 months later: security drift

Images age over time

Naive Debian slim

```
1 trivy image --quiet restapipy:simple | grep "^Total:" -B2
2 restapipy:simple (debian 12.11)
3 =====
4 Total: 106 (UNKNOWN: 1, LOW: 65, MEDIUM: 29, HIGH: 8, CRITICAL: 3)
5 --
6 Python (python-pkg)
7 =====
8 Total: 2 (UNKNOWN: 0, LOW: 0, MEDIUM: 1, HIGH: 1, CRITICAL: 0)
```

Multistage + Alpine runtime

6 months later: security drift

Images age over time

Naive Debian slim

```
1 trivy image --quiet restapipy:simple | grep "^Total:" -B2
2 restapipy:simple (debian 12.11)
3 =====
4 Total: 106 (UNKNOWN: 1, LOW: 65, MEDIUM: 29, HIGH: 8, CRITICAL: 3)
5 --
6 Python (python-pkg)
7 =====
8 Total: 2 (UNKNOWN: 0, LOW: 0, MEDIUM: 1, HIGH: 1, CRITICAL: 0)
```

Multistage + Alpine runtime

```
1 trivy image --quiet restapipy:latest | grep "^Total:" -B2
2 restapipy:latest (alpine 3.22.1)
3 =====
4 Total: 13 (UNKNOWN: 0, LOW: 5, MEDIUM: 7, HIGH: 0, CRITICAL: 1)
5 --
6 Python (python-pkg)
7 =====
8 Total: 2 (UNKNOWN: 0, LOW: 0, MEDIUM: 1, HIGH: 1, CRITICAL: 0)
```

Project structure

Just a boring FastAPI example with uv

```
1 $ tree -I '__pycache__|*.egg-info'
2 .
3 |— Dockerfile           # Multistage Docker build script
4 |— pyproject.toml       # Project metadata and dependencies
5 |— README.md           # Project documentation
6 |— uv.lock              # Locked dependency versions
7 |— src                  # Source code directory
8     |— restapipy
9         |— __init__.py   # Marks the directory as a Python package
10        |— __main__.py   # Entry point for uvicorn
11        |— main.py       # FastAPI application logic
12        |— py.typed       # Indicates type annotations are present
```


FastAPI endpoint + artifact

One endpoint + a build-time OpenAPI spec

```
1 import json
2 from fastapi import FastAPI
3
4 app = FastAPI()
5
6
7 @app.get("/pythonic_breakfast")
8 def _() -> str:
9     return "spam ham eggs"
10
11
12 def generate_openapi_json() -> None:
13     openapi_dict = app.openapi()
14     openapi_json = json.dumps(openapi_dict, indent=4)
15
16     with open("openapi.json", "w") as f:
17         f.write(openapi_json)
```

Multistage Concept

One Dockerfile, multiple containers

Example:

Multistage Concept

One Dockerfile, multiple containers

Example:

- **Builder:** create venv + compile/produce artifacts

Multistage Concept

One Dockerfile, multiple containers

Example:

- **Builder:** create venv + compile/produce artifacts
- **runtime:** copy venv + artifacts, include what's needed

Environment Variables

Some uv optimizations

```
1 ENV UV_LINK_MODE=copy \  
2     UV_COMPILE_BYTECODE=1 \  
3     UV_PYTHON_DOWNLOADS=never
```


Environment Variables

Some uv optimizations

```
1 ENV UV_LINK_MODE=copy \  
2     UV_COMPILE_BYTECODE=1 \  
3     UV_PYTHON_DOWNLOADS=never
```

- **UV_LINK_MODE=copy**

Environment Variables

Some uv optimizations

```
1 ENV UV_LINK_MODE=copy \  
2     UV_COMPILE_BYTECODE=1 \  
3     UV_PYTHON_DOWNLOADS=never
```

- **UV_LINK_MODE=copy**
 - Avoids errors, as UV defaults to hardlinks

Environment Variables

Some uv optimizations

```
1 ENV UV_LINK_MODE=copy \  
2     UV_COMPILE_BYTECODE=1 \  
3     UV_PYTHON_DOWNLOADS=never
```

- **UV_LINK_MODE=copy**
 - Avoids errors, as UV defaults to hardlinks
- **UV_COMPILE_BYTECODE=1**

Environment Variables

Some uv optimizations

```
1 ENV UV_LINK_MODE=copy \  
2     UV_COMPILE_BYTECODE=1 \  
3     UV_PYTHON_DOWNLOADS=never
```

- **UV_LINK_MODE=copy**
 - Avoids errors, as UV defaults to hardlinks
- **UV_COMPILE_BYTECODE=1**
 - Compiles `.py` to `.pyc` during build - speed up startup.

Environment Variables

Some uv optimizations

```
1 ENV UV_LINK_MODE=copy \  
2     UV_COMPILE_BYTECODE=1 \  
3     UV_PYTHON_DOWNLOADS=never
```

- **UV_LINK_MODE=copy**
 - Avoids errors, as UV defaults to hardlinks
- **UV_COMPILE_BYTECODE=1**
 - Compiles `.py` to `.pyc` during build - speed up startup.
- **UV_PYTHON_DOWNLOADS=never**

Environment Variables

Some uv optimizations

```
1 ENV UV_LINK_MODE=copy \  
2     UV_COMPILE_BYTECODE=1 \  
3     UV_PYTHON_DOWNLOADS=never
```

- **UV_LINK_MODE=copy**
 - Avoids errors, as UV defaults to hardlinks
- **UV_COMPILE_BYTECODE=1**
 - Compiles `.py` to `.pyc` during build - speed up startup.
- **UV_PYTHON_DOWNLOADS=never**
 - Prevents `uv` from using Astral compiled Python binaries.

Dockerfile walk-through

Two containers, one Dockerfile

Dockerfile walk-through

Two containers, one Dockerfile

```
1 # builder stage: /app/.venv + artifacts (python:3.13-alpine)
2 FROM python@sha256:610020b9ad8ee92798f1dbe18d5e928d47358db698969d12730f9686
3 COPY --from=ghcr.io/astral-sh/uv:0.8.3 /uv /uvx /bin/
4 WORKDIR /app
5
6 # Sync dependencies first for caching
7 COPY pyproject.toml uv.lock ./
8 RUN uv sync --no-dev --locked --no-editable --no-install-project
9 # Project files will change more often -> separate layers
10 COPY README.md src ./
11 RUN uv sync --no-dev --locked --no-editable
12 # Generate build-time Artifact
13 RUN uv run --no-sync generate_openapi_json
14
15
16 # runtime stage: minimal image (no uv, no compilers, no caches)
17 FROM python@sha256:610020b9ad8ee92798f1dbe18d5e928d47358db698969d12730f9686
18 # Copy pre-built venv + artifacts (nothing else)
19 COPY --from=builder /app/.venv /app/.venv
20 COPY --from=builder /app/openapi.json /app/openapi.json
21 # Don't run as root
```


Dockerfile walk-through

Two containers, one Dockerfile

```
1 # builder stage: /app/.venv + artifacts (python:3.13-alpine)
2 FROM python@sha256:610020b9ad8ee92798f1dbe18d5e928d47358db698969d12730f9686
3 COPY --from=ghcr.io/astral-sh/uv:0.8.3 /uv /uvx /bin/
4 WORKDIR /app
5
6 # Sync dependencies first for caching
7 COPY pyproject.toml uv.lock ./
8 RUN uv sync --no-dev --locked --no-editable --no-install-project
9 # Project files will change more often -> separate layers
10 COPY README.md src ./
11 RUN uv sync --no-dev --locked --no-editable
12 # Generate build-time Artifact
13 RUN uv run --no-sync generate_openapi_json
14
15
16 # runtime stage: minimal image (no uv, no compilers, no caches)
17 FROM python@sha256:610020b9ad8ee92798f1dbe18d5e928d47358db698969d12730f9686
18 # Copy pre-built venv + artifacts (nothing else)
19 COPY --from=builder /app/.venv /app/.venv
20 COPY --from=builder /app/openapi.json /app/openapi.json
21 # Don't run as root
```

Dockerfile walk-through

Two containers, one Dockerfile

```
1 # builder stage: /app/.venv + artifacts (python:3.13-alpine)
2 FROM python@sha256:610020b9ad8ee92798f1dbe18d5e928d47358db698969d12730f9686
3 COPY --from=ghcr.io/astral-sh/uv:0.8.3 /uv /uvx /bin/
4 WORKDIR /app
5
6 # Sync dependencies first for caching
7 COPY pyproject.toml uv.lock ./
8 RUN uv sync --no-dev --locked --no-editable --no-install-project
9 # Project files will change more often -> separate layers
10 COPY README.md src ./
11 RUN uv sync --no-dev --locked --no-editable
12 # Generate build-time Artifact
13 RUN uv run --no-sync generate_openapi_json
14
15
16 # runtime stage: minimal image (no uv, no compilers, no caches)
17 FROM python@sha256:610020b9ad8ee92798f1dbe18d5e928d47358db698969d12730f9686
18 # Copy pre-built venv + artifacts (nothing else)
19 COPY --from=builder /app/.venv /app/.venv
20 COPY --from=builder /app/openapi.json /app/openapi.json
21 # Don't run as root
```


Dockerfile walk-through

Two containers, one Dockerfile

```
1 # builder stage: /app/.venv + artifacts (python:3.13-alpine)
2 FROM python@sha256:610020b9ad8ee92798f1dbe18d5e928d47358db698969d12730f9686
3 COPY --from=ghcr.io/astral-sh/uv:0.8.3 /uv /uvx /bin/
4 WORKDIR /app
5
6 # Sync dependencies first for caching
7 COPY pyproject.toml uv.lock ./
8 RUN uv sync --no-dev --locked --no-editable --no-install-project
9 # Project files will change more often -> separate layers
10 COPY README.md src ./
11 RUN uv sync --no-dev --locked --no-editable
12 # Generate build-time Artifact
13 RUN uv run --no-sync generate_openapi_json
14
15
16 # runtime stage: minimal image (no uv, no compilers, no caches)
17 FROM python@sha256:610020b9ad8ee92798f1dbe18d5e928d47358db698969d12730f9686
18 # Copy pre-built venv + artifacts (nothing else)
19 COPY --from=builder /app/.venv /app/.venv
20 COPY --from=builder /app/openapi.json /app/openapi.json
21 # Don't run as root
```


Dockerfile walk-through

Two containers, one Dockerfile

```
2 FROM python@sha256:610020b9ad8ee92798f1dbe18d5e928d47358db698969d12730f9686
3 COPY --from=ghcr.io/astral-sh/uv:0.8.3 /uv /uvx /bin/
4 WORKDIR /app
5
6 # Sync dependencies first for caching
7 COPY pyproject.toml uv.lock ./
8 RUN uv sync --no-dev --locked --no-editable --no-install-project
9 # Project files will change more often -> separate layers
10 COPY README.md src ./
11 RUN uv sync --no-dev --locked --no-editable
12 # Generate build-time Artifact
13 RUN uv run --no-sync generate_openapi_json
14
15
16 # runtime stage: minimal image (no uv, no compilers, no caches)
17 FROM python@sha256:610020b9ad8ee92798f1dbe18d5e928d47358db698969d12730f9686
18 # Copy pre-built venv + artifacts (nothing else)
19 COPY --from=builder /app/.venv /app/.venv
20 COPY --from=builder /app/openapi.json /app/openapi.json
21 # Don't run as root
22 RUN adduser -D -H worker
23 USER worker
```

Dockerfile walk-through

Two containers, one Dockerfile

```
6 # Sync dependencies first for caching
7 COPY pyproject.toml uv.lock ./
8 RUN uv sync --no-dev --locked --no-editable --no-install-project
9 # Project files will change more often -> separate layers
10 COPY README.md src ./
11 RUN uv sync --no-dev --locked --no-editable
12 # Generate build-time Artifact
13 RUN uv run --no-sync generate_openapi_json
14
15
16 # runtime stage: minimal image (no uv, no compilers, no caches)
17 FROM python@sha256:610020b9ad8ee92798f1dbe18d5e928d47358db698969d12730f9686
18 # Copy pre-built venv + artifacts (nothing else)
19 COPY --from=builder /app/.venv /app/.venv
20 COPY --from=builder /app/openapi.json /app/openapi.json
21 # Don't run as root
22 RUN adduser -D -H worker
23 USER worker
24 EXPOSE 9876
25 ENV PYTHONPATH=/app/.venv/lib/python3.13/site-packages
26 CMD ["python", "-m", "restapipy"]
```

Dockerfile walk-through

Two containers, one Dockerfile

```
6 # Sync dependencies first for caching
7 COPY pyproject.toml uv.lock ./
8 RUN uv sync --no-dev --locked --no-editable --no-install-project
9 # Project files will change more often -> separate layers
10 COPY README.md src ./
11 RUN uv sync --no-dev --locked --no-editable
12 # Generate build-time Artifact
13 RUN uv run --no-sync generate_openapi_json
14
15
16 # runtime stage: minimal image (no uv, no compilers, no caches)
17 FROM python@sha256:610020b9ad8ee92798f1dbe18d5e928d47358db698969d12730f9686
18 # Copy pre-built venv + artifacts (nothing else)
19 COPY --from=builder /app/.venv /app/.venv
20 COPY --from=builder /app/openapi.json /app/openapi.json
21 # Don't run as root
22 RUN adduser -D -H worker
23 USER worker
24 EXPOSE 9876
25 ENV PYTHONPATH=/app/.venv/lib/python3.13/site-packages
26 CMD ["python", "-m", "restapipy"]
```

Dockerfile walk-through

Two containers, one Dockerfile

```
6 # Sync dependencies first for caching
7 COPY pyproject.toml uv.lock ./
8 RUN uv sync --no-dev --locked --no-editable --no-install-project
9 # Project files will change more often -> separate layers
10 COPY README.md src ./
11 RUN uv sync --no-dev --locked --no-editable
12 # Generate build-time Artifact
13 RUN uv run --no-sync generate_openapi_json
14
15
16 # runtime stage: minimal image (no uv, no compilers, no caches)
17 FROM python@sha256:610020b9ad8ee92798f1dbe18d5e928d47358db698969d12730f9686
18 # Copy pre-built venv + artifacts (nothing else)
19 COPY --from=builder /app/.venv /app/.venv
20 COPY --from=builder /app/openapi.json /app/openapi.json
21 # Don't run as root
22 RUN adduser -D -H worker
23 USER worker
24 EXPOSE 9876
25 ENV PYTHONPATH=/app/.venv/lib/python3.13/site-packages
26 CMD ["python", "-m", "restapipy"]
```

Dockerfile walk-through

Two containers, one Dockerfile

```
6 # Sync dependencies first for caching
7 COPY pyproject.toml uv.lock ./
8 RUN uv sync --no-dev --locked --no-editable --no-install-project
9 # Project files will change more often -> separate layers
10 COPY README.md src ./
11 RUN uv sync --no-dev --locked --no-editable
12 # Generate build-time Artifact
13 RUN uv run --no-sync generate_openapi_json
14
15
16 # runtime stage: minimal image (no uv, no compilers, no caches)
17 FROM python@sha256:610020b9ad8ee92798f1dbe18d5e928d47358db698969d12730f9686
18 # Copy pre-built venv + artifacts (nothing else)
19 COPY --from=builder /app/.venv /app/.venv
20 COPY --from=builder /app/openapi.json /app/openapi.json
21 # Don't run as root
22 RUN adduser -D -H worker
23 USER worker
24 EXPOSE 9876
25 ENV PYTHONPATH=/app/.venv/lib/python3.13/site-packages
26 CMD ["python", "-m", "restapipy"]
```


Reproducibility: Digests vs. Tags

Tags can move. Digests don't.

But how do we update digests?

- By hand (manageable in small deployments)
- Tooling e.g. renovate
- Define variables in Dockerfile and pass via `--build-arg`

Example:

```
1 FROM "${CUDA_IMAGE}:${CUDA_VERSION}-cudnn${CUDNN_VERSION}-devel-${OS}" as o
```

Networking safety note (Docker + firewalls)

Don't accidentally expose your service

Networking safety note (Docker + firewalls)

Don't accidentally expose your service

- Docker + firewalls can interact in surprising ways

Networking safety note (Docker + firewalls)

Don't accidentally expose your service

- Docker + firewalls can interact in surprising ways
- Docker creates `iptables` rules, possibly bypassing firewalls like `ufw` or `firewalld`

Networking safety note (Docker + firewalls)

Don't accidentally expose your service

- Docker + firewalls can interact in surprising ways
- Docker creates `iptables` rules, possibly bypassing firewalls like `ufw` or `firewalld`
- Safer defaults:

Networking safety note (Docker + firewalls)

Don't accidentally expose your service

- Docker + firewalls can interact in surprising ways
- Docker creates `iptables` rules, possibly bypassing firewalls like `ufw` or `firewalld`
- Safer defaults:
 - bind to localhost

Networking safety note (Docker + firewalls)

Don't accidentally expose your service

- Docker + firewalls can interact in surprising ways
- Docker creates `iptables` rules, possibly bypassing firewalls like `ufw` or `firewalld`
- Safer defaults:
 - bind to localhost
 - run behind a proxy in deployment

When to change the approach?

Alternative base image

When to change the approach?

Alternative base image

- Alpine uses `musl` instead of `glibc`

When to change the approach?

Alternative base image

- Alpine uses `musl` instead of `glibc`
 - can impact performance

When to change the approach?

Alternative base image

- Alpine uses `musl` instead of `glibc`
 - can impact performance
 - surprises with native extensions

When to change the approach?

Alternative base image

- Alpine uses `musl` instead of `glibc`
 - can impact performance
 - surprises with native extensions
- Do you need cuda?

Takeaways

Takeaways

- Multi-stage builds: no more build tools in runtime images

Takeaways

- Multi-stage builds: no more build tools in runtime images
- Layering: copy lock files first -> faster rebuilds

Takeaways

- Multi-stage builds: no more build tools in runtime images
- Layering: copy lock files first -> faster rebuilds
- `uv sync + uv.lock`: fast and reproducible

Takeaways

- Multi-stage builds: no more build tools in runtime images
- Layering: copy lock files first -> faster rebuilds
- `uv sync + uv.lock`: fast and reproducible
- Pin by digest for reproducibility

Takeaways

- Multi-stage builds: no more build tools in runtime images
- Layering: copy lock files first -> faster rebuilds
- `uv sync` + `uv.lock`: fast and reproducible
- Pin by digest for reproducibility
- Alpine is fine for “small & minimal”, but Debian/Ubuntu can be “less pain”

References

- Original Blog Post - https://digon.io/en/blog/2025_07_28_python_docker_images_with_uv
- Itamar Turner-Trauring - <https://pythonspeed.com/articles/uv-python-production/>
- Hynek Schlawack - <https://hynek.me/articles/docker-uv/>

Thank you 🖐️

Questions?

Thank you 🙌

Questions?

